SmartDoc AI - Intelligent Document Q&A System

Huỳnh Lê Anh Tuấn*, Nguyễn Võ Anh Kỳ*, Lương Mai Hoàng Văn*,
Ngô Kiến Văn*, Nguyễn Thanh Văn*, Hồ Quốc Đạt*
*Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn
Email: {hlatuan0793, kyy07072005, hv15032005, ngokienvan2005, thanhvan13131, hahodat1958}@gmail.com

***Tóm tắt nội dung*—Dự án SmartDoc AI tập trung nghiên cứu và xây dựng hệ thống hỏi đáp tài liệu thông minh dựa trên kiến trúc Retrieval-Augmented Generation (RAG). Trong quá trình thực hiện, nhóm đã tìm hiểu và áp dụng các công nghệ cốt lõi như mô hình ngôn ngữ lớn Qwen2.5, kỹ thuật biểu diễn văn bản đa ngôn ngữ (Multilingual Embedding với MPNet) và cơ sở dữ liệu vector FAISS.**

Bên cạnh đó, pipeline xử lý dữ liệu được thiết kế hoàn chỉnh, bao gồm trích xuất nội dung từ file PDF bằng PDFPlumber, phân đoạn tài liệu (chunking), lập chỉ mục và truy xuất thông tin. Kỹ năng lập trình được củng cố thông qua việc sử dụng framework LangChain để xây dựng hệ thống và Streamlit để phát triển giao diện người dùng. Ngoài ra, kỹ thuật Prompt Engineering được áp dụng nhằm tối ưu hóa khả năng phản hồi, đặc biệt với tiếng Việt.

Kết quả đạt được cho thấy hệ thống có thể vận hành hiệu quả trên môi trường cục bộ, đảm bảo tính bảo mật và cung cấp câu trả lời chính xác dựa trên tài liệu đầu vào.

***Keywords*-RAG; Qwen2.5; FAISS; LangChain; Streamlit; Prompt Engineering; Hỏi đáp tài liệu**

I. Giới thiệu và công trình liên quan

A. Bối cảnh và động lực

Trong kỷ nguyên số, khối lượng tài liệu điện tử như PDF và DOCX ngày càng gia tăng mạnh mẽ trong các lĩnh vực học tập, nghiên cứu và doanh nghiệp. Việc trích xuất, tìm kiếm và khai thác thông tin từ các tài liệu này trở thành một nhu cầu thiết yếu. Tuy nhiên, các phương pháp tìm kiếm truyền thống dựa trên từ khóa thường gặp nhiều hạn chế do không thể hiểu được ngữ nghĩa và ngữ cảnh thực sự của nội dung, dẫn đến kết quả truy xuất chưa chính xác.

Các hệ thống *Retrieval-Augmented Generation* (RAG) thông thường chủ yếu dựa trên phương pháp truy xuất bằng độ tương đồng vector (*vector similarity*), giúp cải thiện đáng kể khả năng tìm kiếm ngữ nghĩa. Tuy vậy, mô hình này vẫn còn hạn chế khi xử lý các câu hỏi phức tạp yêu cầu suy luận nhiều bước hoặc cần hiểu rõ mối quan hệ giữa các thực thể trong tài liệu.

Xuất phát từ thực tế đó, dự án SmartDoc AI được xây dựng nhằm nâng cao hiệu quả truy xuất và hỏi đáp tài liệu bằng cách ứng dụng kiến trúc RAG kết hợp Hybrid Search (FAISS + BM25), Query Rewriting, Self-RAG Validation và Reranking. Hệ thống sử dụng LangChain để quản lý toàn bộ pipeline xử lý, FAISS làm cơ sở dữ liệu vector, BM25 cho truy xuất từ khóa truyền thống, và Ollama để triển khai mô hình *Qwen2.5:7B* chạy cục bộ. Giải pháp này giúp tăng độ chính xác của câu trả lời, cải thiện khả năng hiểu ngữ cảnh và đảm bảo tính bảo mật dữ liệu nội bộ.

B. Mục tiêu dự án

Dự án hướng đến việc xây dựng một hệ thống hỏi đáp tài liệu thông minh dựa trên kiến trúc *Retrieval-Augmented Generation* (RAG), nhằm nâng cao khả năng truy xuất và khai thác thông tin từ các tài liệu PDF/DOCX. Các mục tiêu chính bao gồm:

- Xây dựng pipeline hoàn chỉnh từ quá trình tải tài liệu, tiền xử lý, chia nhỏ văn bản (chunking), tạo vector embedding, lưu trữ dữ liệu cho đến sinh câu trả lời cuối cùng.
- Áp dụng mô hình embedding đa ngôn ngữ kết hợp với FAISS và BM25 để triển khai cơ chế Hybrid Search, giúp nâng cao hiệu quả tìm kiếm cả về ngữ nghĩa lẫn từ khóa.
- Sử dụng mô hình *Qwen2.5:7B* chạy cục bộ thông qua Ollama nhằm đảm bảo khả năng suy luận tốt, giảm phụ thuộc vào dịch vụ bên ngoài và tăng tính bảo mật dữ liệu.
- Tích hợp cơ chế Query Rewriting, Self-RAG Validation và Reranking nhằm cải thiện chất lượng truy xuất và độ chính xác của câu trả lời.
- Phát triển giao diện người dùng trực quan bằng Streamlit, hỗ trợ tải tài liệu, đặt câu hỏi và hiển thị kết quả một cách thuận tiện.

C. Kiến thức tiếp thu

Trong quá trình thực hiện dự án, nhóm đã tiếp cận và củng cố nhiều kiến thức quan trọng liên quan đến lĩnh vực xử lý ngôn ngữ tự nhiên và hệ thống hỏi đáp thông minh. Các nội dung chính bao gồm:

- Hiểu rõ kiến trúc *Retrieval-Augmented Generation* (RAG), cơ chế phối hợp giữa bộ truy xuất thông tin (*Retriever*) và mô hình sinh phản hồi (*Generator*) trong việc xây dựng hệ thống hỏi đáp tài liệu.
- Nắm vững các kỹ thuật xử lý ngôn ngữ tự nhiên (NLP) như chia nhỏ văn bản (*chunking*), tạo chỉ mục (*indexing*), vector hóa dữ liệu (*embedding*) và tìm kiếm ngữ nghĩa (*semantic search*).
- Tìm hiểu và áp dụng phương pháp Hybrid Search thông qua việc kết hợp FAISS và BM25 nhằm nâng cao độ chính xác trong truy xuất thông tin.
- Nghiên cứu kỹ thuật *Prompt Engineering* để tối ưu chất lượng phản hồi từ mô hình ngôn ngữ lớn, bao gồm Query Rewriting, Self-RAG Validation và Multi-hop Retrieval.
- Sử dụng LangChain để xây dựng và điều phối toàn bộ pipeline RAG, đồng thời triển khai mô hình *Qwen2.5:7B* chạy cục bộ thông qua Ollama nhằm đảm bảo hiệu năng và tính bảo mật dữ liệu.

D. Công trình liên quan

Dự án tham khảo các công trình nghiên cứu và tài liệu kỹ thuật tiêu biểu sau:

- Kiến trúc *Retrieval-Augmented Generation* (RAG) của Lewis và cộng sự [6], đặt nền tảng cho việc kết hợp giữa truy xuất thông tin và mô hình ngôn ngữ lớn nhằm nâng cao chất lượng câu trả lời.
- Thư viện FAISS (*Facebook AI Similarity Search*) của Johnson và cộng sự [3], hỗ trợ tìm kiếm tương đồng vector hiệu quả trên tập dữ liệu lớn.
- Kiến trúc *Transformer* và hệ sinh thái HuggingFace Transformers của Wolf và cộng sự [5], phục vụ cho quá trình embedding và xử lý ngôn ngữ tự nhiên.
- Tài liệu chính thức của LangChain [1], cung cấp framework để xây dựng và điều phối toàn bộ pipeline RAG.
- Tài liệu hướng dẫn từ Streamlit [4], hỗ trợ phát triển giao diện người dùng trực quan cho hệ thống hỏi đáp tài liệu.
- Nền tảng Ollama [2], được sử dụng để triển khai mô hình *Qwen2.5:7B* chạy cục bộ, đảm bảo hiệu năng và tính bảo mật dữ liệu.

II. Thực nghiệm

A. Thiết lập môi trường và dữ liệu

Hệ thống được triển khai trên môi trường cục bộ (*Windows/Linux*) với Python 3.9+ nhằm đảm bảo khả năng tương thích với các thư viện xử lý ngôn ngữ tự nhiên và truy xuất dữ liệu. Mô hình ngôn ngữ lớn được vận hành thông qua Ollama, cho phép chạy cục bộ mô hình *Qwen2.5:7B*, giúp tăng tính bảo mật và giảm phụ thuộc vào các dịch vụ đám mây bên ngoài.

Các thư viện chính được sử dụng bao gồm LangChain để xây dựng pipeline RAG, FAISS cho truy xuất vector ngữ nghĩa, BM25 cho truy xuất từ khóa truyền thống, Sentence-Transformers cho embedding và reranking, cùng Streamlit để phát triển giao diện người dùng.

Dữ liệu thử nghiệm bao gồm nhiều tệp PDF và DOCX thuộc các lĩnh vực khác nhau như tài liệu kỹ thuật, báo cáo chuyên ngành và bài báo khoa học. Các tập dữ liệu này được lựa chọn nhằm đánh giá khả năng trích xuất nội dung, tìm kiếm ngữ nghĩa và chất lượng phản hồi của hệ thống trong nhiều tình huống truy vấn khác nhau.

B. Cấu hình hệ thống

Pipeline xử lý dữ liệu được thiết kế với các tham số chính như sau:

- **Chunking:** sử dụng `chunk_size = 1000` và `chunk_overlap = 100`, có thể điều chỉnh để đảm bảo mỗi đoạn văn bản giữ được đầy đủ ngữ cảnh nhưng vẫn tối ưu cho quá trình truy xuất.
- **Embedding Model:** sử dụng mô hình *sentence-transformers/paraphrase-multilingual-mpnet-base-v2* với vector biểu diễn 768 chiều, hỗ trợ đa ngôn ngữ và được tối ưu tốt cho tiếng Việt.
- **Vector Database:** sử dụng FAISS (*Facebook AI Similarity Search*) để lưu trữ vector embedding và thực hiện tìm kiếm tương đồng ngữ nghĩa với hiệu năng cao.
- **Sparse Retrieval:** kết hợp BM25 (*Best Matching 25*) nhằm tăng độ chính xác trong truy xuất từ khóa truyền thống, tạo thành mô hình Hybrid Search cùng với FAISS.
- **Reranking Model:** sử dụng mô hình *cross-encoder/ms-marco-MiniLM-L-6-v2* để sắp xếp

lại các kết quả truy xuất, giúp lựa chọn ngữ cảnh phù hợp nhất trước khi đưa vào LLM.

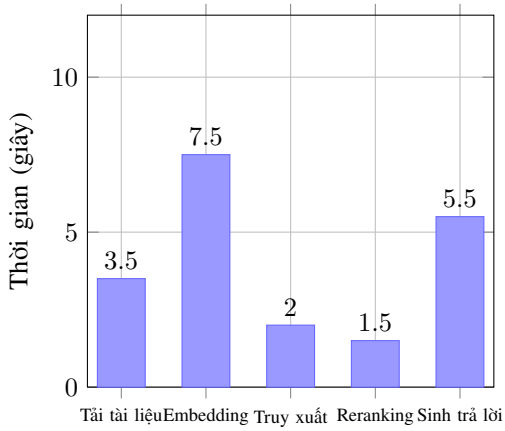
- **LLM:** sử dụng mô hình *Qwen2.5:7B* chạy cục bộ thông qua Ollama, đảm bảo khả năng suy luận tốt và bảo mật dữ liệu nội bộ.
- **Framework:** hệ thống được xây dựng trên nền tảng LangChain kết hợp Streamlit để triển khai giao diện người dùng và điều phối toàn bộ pipeline RAG.

C. Hiệu suất hệ thống

Bảng I
THỜI GIAN XỬ LÝ TRUNG BÌNH CỦA HỆ THỐNG

Giai đoạn	Thời gian (giây)
Tải tài liệu PDF/DOCX	2–5
Tạo Embedding (100 chunks)	5–10
Truy xuất thông tin (FAISS + BM25)	1–3
Reranking kết quả truy xuất	1–2
Sinh câu trả lời (LLM)	3–8

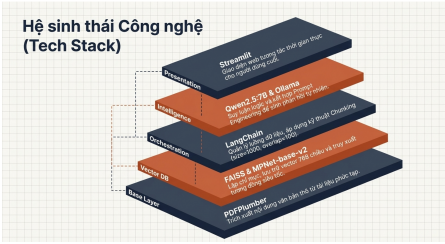
Bảng I cho thấy các tác vụ tạo embedding và sinh câu trả lời chiếm phần lớn thời gian xử lý do liên quan trực tiếp đến quá trình tính toán của mô hình AI và mô hình ngôn ngữ lớn. Trong khi đó, bước truy xuất thông tin từ FAISS kết hợp BM25 diễn ra khá nhanh nhờ cơ chế tối ưu hóa tìm kiếm vector và truy xuất từ khóa truyền thống. Tổng thời gian phản hồi của hệ thống vẫn đảm bảo tính tương tác tốt, phù hợp cho các ứng dụng hỏi đáp tài liệu thời gian thực.



Hình 1. Kết quả đo thời gian xử lý các giai đoạn trong hệ thống

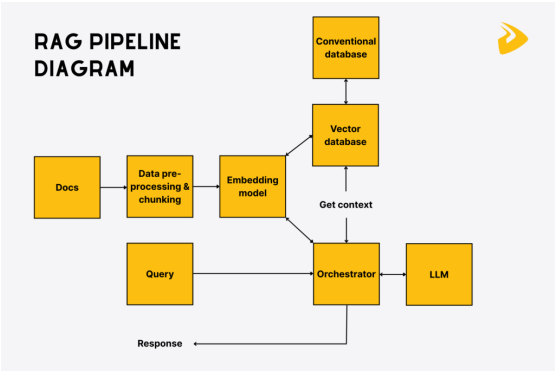
Hình 1 minh họa trực quan sự khác biệt về thời gian giữa các giai đoạn xử lý trong pipeline RAG. Có thể thấy bước tạo embedding và sinh câu trả lời tiêu tốn nhiều tài nguyên nhất do yêu cầu xử lý mô hình học sâu và suy luận ngôn ngữ. Ngược lại, quá trình truy xuất từ FAISS kết hợp BM25 diễn ra nhanh chóng, góp phần đảm bảo tốc độ phản hồi tổng thể của hệ thống.

D. Minh họa hệ thống



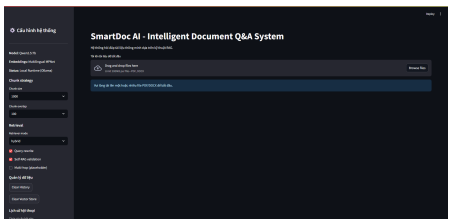
Hình 2. Kiến trúc tổng thể của hệ thống SmartDoc AI

Hình 2 mô tả kiến trúc tổng thể của hệ thống SmartDoc AI, bao gồm các thành phần chính như giao diện người dùng, bộ xử lý tài liệu, hệ thống truy xuất thông tin và mô hình ngôn ngữ lớn. Kiến trúc này được thiết kế theo hướng mô-đun nhằm đảm bảo khả năng mở rộng, dễ bảo trì và tối ưu hiệu suất xử lý.



Hình 3. Pipeline xử lý của hệ thống SmartDoc AI

Như thể hiện trong Hình 3, hệ thống được xây dựng dựa trên kiến trúc *Retrieval-Augmented Generation* (RAG) với quy trình gồm các bước chính: tải tài liệu đầu vào, tiền xử lý và chia nhỏ văn bản (chunking), tạo vector embedding, lưu trữ trong FAISS kết hợp với BM25 để thực hiện Hybrid Search, sau đó áp dụng Reranking và đưa ngữ cảnh phù hợp vào mô hình ngôn ngữ lớn để sinh câu trả lời cuối cùng. Pipeline này giúp nâng cao độ chính xác trong truy xuất và đảm bảo phản hồi có tính ngữ cảnh cao.



Hình 4. Giao diện người dùng của hệ thống trên nền tảng Streamlit

Hình 4 minh họa giao diện người dùng được phát triển bằng Streamlit, cho phép người dùng dễ dàng tải lên tài liệu PDF/DOCX, nhập câu hỏi và nhận phản hồi trực tiếp từ hệ thống. Giao diện được thiết kế đơn giản, trực quan và thân thiện, hỗ trợ quá trình tương tác nhanh chóng và thuận tiện trong quá trình sử dụng.

E. Tích hợp GraphRAG

Bên cạnh kiến trúc RAG truyền thống sử dụng Vector Database để truy xuất ngữ nghĩa, hệ thống được mở rộng bằng cách tích hợp GraphRAG nhằm nâng cao khả năng xử lý các truy vấn phức tạp, đặc biệt là các câu hỏi yêu cầu suy luận nhiều bước (multi-hop reasoning). GraphRAG kết hợp giữa truy xuất ngữ nghĩa và truy xuất dựa trên cấu trúc quan hệ, giúp hệ thống không chỉ tìm kiếm các đoạn văn bản tương đồng mà còn khai thác được mối liên hệ giữa các thực thể trong tài liệu.

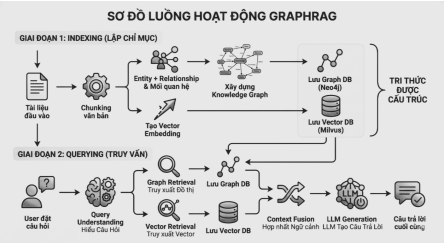
Quy trình hoạt động của GraphRAG bắt đầu từ giai đoạn tiền xử lý tài liệu. Sau khi tài liệu được tải lên và phân chia thành các đoạn nhỏ (chunking), hệ thống tiến hành trích xuất thực thể (Entity Extraction) và mối quan hệ giữa các thực thể đó (Relationship Extraction). Các thực thể có thể bao gồm tên người, tổ chức, sản phẩm, công nghệ, khái niệm chuyên ngành hoặc các thành phần kỹ thuật quan trọng trong tài liệu. Từ đó, hệ thống xây dựng Knowledge Graph dưới dạng các node và edge, trong đó node đại diện cho thực thể và edge biểu diễn mối quan hệ giữa chúng.

Song song với quá trình xây dựng đồ thị tri thức, các đoạn văn bản vẫn được chuyển đổi thành vector embedding và lưu trữ trong FAISS để phục vụ truy xuất ngữ nghĩa như mô hình RAG truyền thống. Việc kết hợp cả hai phương pháp giúp tận dụng ưu điểm của từng mô hình: Vector Search hỗ trợ tìm kiếm ngữ nghĩa nhanh chóng, trong khi Graph Retrieval hỗ trợ truy vết quan hệ và suy luận logic.

Khi người dùng đặt câu hỏi, hệ thống trước tiên phân tích truy vấn để xác định các thực thể liên quan và đánh giá xem câu hỏi có yêu cầu suy luận nhiều bước hay không. Nếu truy vấn liên quan đến nhiều thực thể hoặc yêu cầu truy xuất theo chuỗi quan hệ, hệ thống sẽ thực hiện Graph Traversal trên Knowledge Graph để tìm các đường đi liên quan nhất. Đồng thời, Vector Retrieval vẫn được sử dụng để lấy các đoạn văn bản bổ sung nhằm tăng độ đầy đủ của ngữ cảnh.

Kết quả từ Graph Retrieval và Vector Retrieval sau đó được hợp nhất (Context Fusion) để tạo thành ngữ cảnh đầu vào tối ưu cho mô hình ngôn ngữ lớn (LLM). Nhờ đó, câu trả lời được sinh ra không chỉ chính xác hơn mà còn có khả năng giải thích rõ ràng hơn dựa trên các mối quan hệ được truy vết trong đồ thị.

Việc tích hợp GraphRAG giúp hệ thống cải thiện đáng kể hiệu quả xử lý các tài liệu chuyên sâu như báo cáo nghiên cứu, tài liệu kỹ thuật, hướng dẫn sử dụng và tài liệu doanh nghiệp có cấu trúc thông tin phức tạp. So với RAG truyền thống, GraphRAG giảm hiện tượng bỏ sót ngữ cảnh, tăng khả năng giải thích kết quả và nâng cao độ chính xác đối với các truy vấn mang tính suy luận logic.



Hình 5. Sơ đồ tích hợp GraphRAG trong hệ thống SmartDoc AI

III. Hướng phát triển

Dựa trên kết quả thực nghiệm và phân tích hiệu suất của hệ thống, các hướng phát triển tiếp theo tập trung vào việc nâng cao chất lượng truy xuất, cải thiện độ chính xác của câu trả lời và mở rộng hạ tầng triển khai. Bên cạnh đó, hệ thống cũng hướng đến khả năng tích hợp các nền tảng điện toán đám mây nhằm tăng tính mở rộng và tối ưu hiệu năng xử lý.

A. Cải thiện ngắn hạn

- **Tích hợp Self-RAG:** Phát triển cơ chế tự đánh giá câu trả lời của mô hình ngôn ngữ lớn nhằm nâng cao độ tin cậy của kết quả đầu ra. Self-RAG cho phép hệ thống tự kiểm tra mức độ liên quan (*relevance*) và tính chính xác của câu trả lời trước khi phản hồi cho người dùng.

- **Query Rewriting:** Xây dựng module tự động cải thiện và mở rộng câu hỏi đầu vào của người dùng nhằm tăng hiệu quả truy xuất thông tin. Quá trình này bao gồm *query expansion*, *paraphrasing* và *context enrichment*, giúp hệ thống hiểu rõ hơn ý định truy vấn.
- **Tối ưu kích thước Chunking:** Tự động điều chỉnh kích thước chunk và số lượng đoạn văn bản được truy xuất nhằm cân bằng giữa chất lượng ngữ cảnh và tốc độ xử lý. Việc tối ưu chunking giúp giảm chi phí tính toán và nâng cao hiệu quả của quá trình retrieval.
- **Cải thiện Reranking:** Nâng cấp mô hình reranking nhằm lựa chọn chính xác hơn các đoạn văn bản phù hợp nhất trước khi đưa vào LLM, từ đó cải thiện chất lượng phản hồi cuối cùng.

B. Cải thiện dài hạn

- **Tích hợp hạ tầng Cloud (AWS):**
  - **Amazon Bedrock:** Sử dụng các Foundation Models như Claude hoặc Titan để nâng cao chất lượng sinh phản hồi và mở rộng khả năng xử lý đa tác vụ.
  - **Amazon Neptune:** Mở rộng sang mô hình Graph RAG bằng cách triển khai Knowledge Graph trên Neptune nhằm hỗ trợ suy luận đa bước và truy xuất quan hệ giữa các thực thể hiệu quả hơn.
  - **Amazon DMS:** Xây dựng pipeline đồng bộ dữ liệu tự động từ nhiều nguồn khác nhau, hỗ trợ cập nhật dữ liệu liên tục cho hệ thống.
  - **Langfuse:** Tích hợp công cụ giám sát LLM (*LLM Observability*) để theo dõi, phân tích và tối ưu các lời gọi mô hình ngôn ngữ lớn.
- **Triển khai Self-RAG hoàn chỉnh:** Nghiên cứu sâu hơn về cơ chế *Self-reflection* để mô hình có thể tự kiểm chứng độ tin cậy của tài liệu truy xuất cũng như tính chính xác của câu trả lời trước khi phản hồi.
- **Advanced Query Rewriting:** Phát triển mô hình chuyển đổi truy vấn (*Query Transformation*) nhằm tự động biến các câu hỏi mơ hồ thành các truy vấn tối ưu hơn cho hệ thống retrieval, từ đó nâng cao tỷ lệ tìm kiếm chính xác.
- **Mở rộng đa định dạng tài liệu:** Hỗ trợ thêm nhiều định dạng dữ liệu như Excel, PowerPoint, HTML và dữ liệu từ cơ sở dữ liệu doanh nghiệp nhằm mở rộng phạm vi ứng dụng thực tế của hệ thống.

IV. Tài liệu tham khảo

Tài liệu

[1] LangChain, "LangChain: Building applications with LLMs through composability," [Online]. Available: https://python.langchain.com/

[2] Ollama, "Get up and running with large language models locally," [Online]. Available: https://ollama.com/

[3] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, 2019.

[4] Streamlit, "The fastest way to build and share data apps," [Online]. Available: https://docs.streamlit.io/

[5] T. Wolf *et al.*, "Transformers: State-of-the-art Natural Language Processing," in *Proc. EMNLP*, 2020.

[6] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. NeurIPS*, 2020.